# Q3. Handling Missing Values

Original Aim: To demonstrate missing-value handling using the Iris dataset. Create a working copy and introduce missing values in a few cells of a numerical column and a categorical column. Detect the missing values, fill numerical gaps with the column median and categorical gaps with the mode, and verify that no missing values remain in those columns.

## Aim

Introduce controlled missing values on a working copy, fill numerical gaps with a median and categorical gaps with a mode, and verify the result.

## Requirements

Python, NumPy, Pandas and scikit-learn. No missing values are assumed to exist in the original bundled data.

## Essential theory

NaN represents a missing value. `isna()` detects it. The median is the middle observed numerical value and is less affected by extreme observations than the mean. A mode is a most frequent category; a numerical mean is not meaningful for species names. Simple imputation fills cells but does not recover their unknown true values.

## Step-by-step procedure

1. Construct the original DataFrame and create a deep working copy.
2. Set SepalLengthCm at row indices 0, 10 and 20 to NaN.
3. Set Species at row indices 0, 50 and 51 to NaN.
4. Count missing cells and display all affected rows.
5. Calculate the median and mode from the remaining observed values in the working copy.
6. Fill each column with its appropriate statistic, then verify zero missing values and an unchanged complete original.

## Complete Python program

```
import numpy as np
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)

work = df.copy(deep=True)
num_rows = [0, 10, 20]
cat_rows = [0, 50, 51]
work.loc[num_rows, "SepalLengthCm"] = np.nan
work.loc[cat_rows, "Species"] = np.nan
cols = ["SepalLengthCm", "Species"]
changed_rows = sorted(set(num_rows + cat_rows))
print("MISSING COUNTS BEFORE")
print(work[cols].isna().sum().to_string())
print("AFFECTED ROWS BEFORE")
print(work.loc[changed_rows, cols].to_string())
median_value = work["SepalLengthCm"].median()
mode_value = work["Species"].mode().iloc[0]
print("Median:", median_value)
print("Mode:", mode_value)
work["SepalLengthCm"] = work["SepalLengthCm"].fillna(median_value)
work["Species"] = work["Species"].fillna(mode_value)
print("AFFECTED ROWS AFTER")
print(work.loc[changed_rows, cols].to_string())
print("MISSING COUNTS AFTER")
print(work[cols].isna().sum().to_string())
print("Original missing count:", int(df.isna().sum().sum()))
assert not work[cols].isna().any().any()
assert not df.isna().any().any()
```

## Code explanation

- `copy(deep=True)` creates a separate table for this exercise. Other practicals use fresh complete data.
- `.loc[rows, column] = np.nan` assigns into explicit row labels without chained assignment.
- `isna().sum()` counts gaps per column. Median and mode normally exclude missing values.
- `mode().iloc[0]` selects the first mode. This controlled example has a unique observed mode, Iris-virginica; if several modes tie, state your chosen tie-breaking rule.
- Assigning the returned `fillna` Series back to its column avoids ambiguous chained inplace updates. The assertions fail if either final verification is false.

## Verified output

```
MISSING COUNTS BEFORE
SepalLengthCm    3
Species          3
AFFECTED ROWS BEFORE
    SepalLengthCm      Species
0             NaN          NaN
10            NaN  Iris-setosa
20            NaN  Iris-setosa
50            7.0          NaN
51            6.4          NaN
Median: 5.8
Mode: Iris-virginica
AFFECTED ROWS AFTER
    SepalLengthCm         Species
0             5.8  Iris-virginica
10            5.8     Iris-setosa
20            5.8     Iris-setosa
50            7.0  Iris-virginica
51            6.4  Iris-virginica
MISSING COUNTS AFTER
SepalLengthCm    0
Species          0
Original missing count: 0
```

## Observation

Each selected column initially has three gaps. The remaining observed sepal lengths have median 5.8 cm. Removing two versicolor labels and one setosa label leaves virginica most frequent, so all three categorical gaps become Iris-virginica. This demonstrates the requested rule; it does not correctly reconstruct the true species for rows 0, 50 and 51. The original table remains complete.

## Result

All introduced gaps were filled and the original data were preserved. In a real supervised task, do not invent missing target labels by mode imputation; obtain reliable labels or exclude unlabelled examples from supervised fitting. Fit feature imputers on training data only when evaluating a model.

## Common errors and troubleshooting

- Comparing a cell with == np.nan: use isna(), because NaN does not compare equal in the ordinary way.
- Mode index error: an entirely missing column has no observed mode; define a justified fallback rather than indexing an empty result.
- Unexpected mutation: create the working copy before introducing gaps and avoid ambiguous chained indexing.

## Viva questions and answers

- Why use a median for numerical gaps? It gives a central observed value and is less sensitive to extremes.
- Why use a mode for Species? Categories have no meaningful arithmetic average.
- Does zero missing count mean correct recovered data? No; it only confirms no cells remain missing.
- Why not calculate the median after filling? The replacement statistic must be calculated from the observed values first.